第 4 章：光线追踪

中文版 · 全章整合

章节目录

	PDF 页码
第 4 章导言：光线追踪	2
4.1 基本光线追踪算法	3
4.2 透视	5
4.3 观察射线与正交视图	7
4.3 透视视图	11
4.4 射线与球体求交	12
4.4 射线与三角形求交	14
4.4 软件中的射线求交	17
4.4 与一组物体求交	18
4.5 着色与光源	19
4.5 软件中的着色	20
4.5.3 阴影	22
4.5.4 镜面反射	24
4.6 历史注释	26
常见问题与习题	27

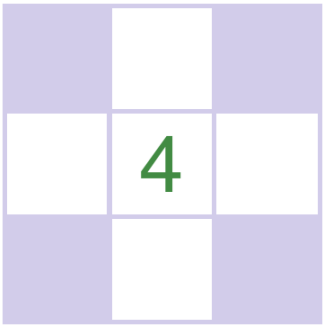
按原书顺序合并各分节译文；原有插图、公式与分节页脚均保留。

点选目录条目或使用阅读器书签，可直接跳转至相应章节。

第 4 章：光线追踪（Ray Tracing）

《Fundamentals of Computer Graphics》第 5 版 • 中文译本 • 第三十六批

翻译范围：原书第 79—80 页的章首导言，截止 4.1 节之前；保留章首图案与两条原书边注



计算机图形学的一项基本任务，是对三维物体进行渲染（rendering）：给定一个由许多几何物体组成的场景，这些物体分布在三维空间中，我们需要计算出一幅二维图像，呈现从某个特定观察位置看过去时，这些物体的样子。数百年来，建筑师和工程师一直通过绘制图纸向他人传达设计方案；他们所做的，本质上就是同一件事。

从根本上说，渲染是一个以一组物体为输入、以一个像素数组为输出的过程。无论采用哪种方法，渲染都需要考虑每一个物体如何影响每一个像素。这个过程大体可以按照两种方式组织。采用按对象顺序渲染（object-order rendering）时，我们依次处理每个物体：针对每个物体，找出它所影响的全部像素，并更新这些像素。采用按图像顺序渲染（image-order rendering）时，我们依次处理每个像素：针对每个像素，找出所有会影响它的物体，再计算该像素的值。

可以从循环嵌套的角度来理解两者的差异：在按图像顺序渲染中，外层循环是“遍历每个像素”；而在按对象顺序渲染中，外层循环是“遍历每个物体”。

原书边注：如果输出的是矢量图像，而非光栅图像，那么渲染过程不一定需要涉及像素。不过，本书将以光栅图像为前提进行讨论。

按图像顺序渲染与按对象顺序渲染，能够计算出完全相同的图像；但两者分别更适合实现不同种类的效果，而且性能特征有很大差异。在分别讨论这两类方法之后，我们会在第 9 章考察它们各自的相对优势。不过，概括地说，按图像顺序渲染通常更容易实现，也更灵活，能够产生的效果种类更多；而要生成一幅效果相当的图像，它所花费的运行时间通常也更长——当然并非总是如此。

原书边注：在光线追踪器中，计算精确的阴影和反射比较容易；但在按对象顺序渲染的框架下，这些效果较难处理。

光线追踪（ray tracing）是一种按照图像顺序组织的算法，用来渲染三维场景。我们会首先讨论这种算法，因为即使尚未建立按对象顺序渲染所需的那些数学工具，也可以先实现一个能够工作的光线追踪器。

第 4 章：光线追踪（4.1）

《Fundamentals of Computer Graphics》第 5 版 • 中文译本 • 第三十七批

翻译范围：原书第 80—81 页 • 保留三阶段算法、完整伪代码与图 4.1

4.1 基本光线追踪算法（The Basic Ray-Tracing Algorithm）

光线追踪器的工作方式，是一次计算一个像素。对于每个像素，最基本的任务就是找出：从图像中这个像素所在的位置，我们能够看到场景中的哪一个物体。每个像素都朝着不同的方向“观察”。凡是能够被某个像素看到的物体，都必须与该像素对应的观察射线（viewing ray）相交。所谓观察射线，就是一条从观察点出发、沿着该像素所观察的方向延伸的直线。

我们真正想找到的物体，是沿观察射线距离相机最近的那个相交物体，因为它会挡住后面其他物体。一旦找到了这个物体，着色（shading）计算便会利用交点位置、表面法线以及其他信息（具体取决于希望实现哪种渲染效果），确定该像素的颜色。图 4.1 展示了这种情况：一条射线虽然与两个三角形相交，但只有首先被击中的三角形 T_2 会接受着色计算。

因此，一个基本的光线追踪器由三个部分组成：

1. 射线生成（ray generation）：根据相机的几何关系，计算每个像素对应的观察射线的起点和方向。
2. 射线求交（ray intersection）：寻找与观察射线相交、且距离最近的物体。
3. 着色（shading）：依据射线求交的结果，计算像素的颜色。

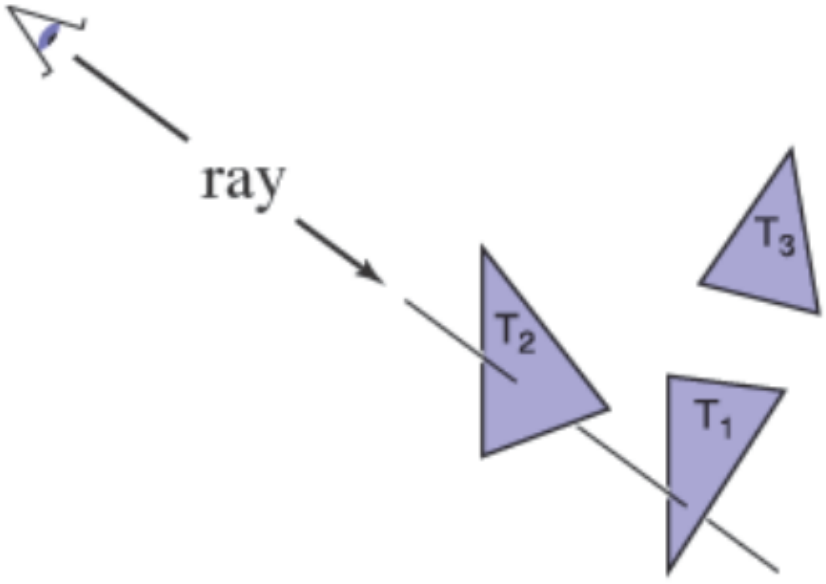


图 4.1 沿着观察射线在场景中进行“追踪”，首先被击中的物体就是该像素看见的物体。本例中，射线返回的是三角形 T_2。

基本光线追踪程序的结构如下：

> 对每一个像素执行：
> 计算观察射线；
> 找出射线首先击中的物体，以及该处的表面法线 n；
> 根据交点、光源和法线 n 计算像素颜色，并将其写入该像素。

本章介绍射线生成、射线求交和着色的基本方法；利用这些方法，已经足以实现一个简单的演示用光线追踪器。不过，要建立一个真正实用的系统，还需要加入第 12 章将介绍的、更高效的射线求交技术。至于光线追踪器的真正潜力，则要结合第 14 章讨论的更高级渲染技术，才能充分展现出来。

第 4 章：光线追踪（4.2）

《Fundamentals of Computer Graphics》第 5 版 • 中文译本 • 第三十八批

翻译范围：原书第 81—82 页 • 保留图 4.2—4.4、全部正文和原书边注

4.2 透视（Perspective）

早在计算机出现之前的数百年，艺术家们就已经开始研究如何用二维绘画表现三维物体或场景。照片同样是用二维图像来表现三维场景。虽然人们可以用许多不常规的方式制作图像，例如立体主义绘画、鱼眼镜头（图 4.2）以及周边视野相机，但无论是在绘画、摄影还是计算机图形学中，标准方法都是线性透视（linear perspective）。在线性透视中，三维物体被投影到一个像平面（image plane）上，并且场景中的直线在图像中仍然表现为直线。



图 4.2 使用鱼眼镜头拍摄的图像，并不是线性透视图像。照片由 Philip Greenspun 提供。

最简单的一种投影是平行投影（parallel projection）：沿着同一个投影方向移动三维点，直到它们到达像平面，从而将三维点映射到二维平面（见图 4.3—4.4）。最终形成的视图取决于投影方向与像平面的选择。如果像平面垂直于观察方向，这种投影称为正交投影（orthographic projection）；否则，便称为斜投影（oblique projection）。

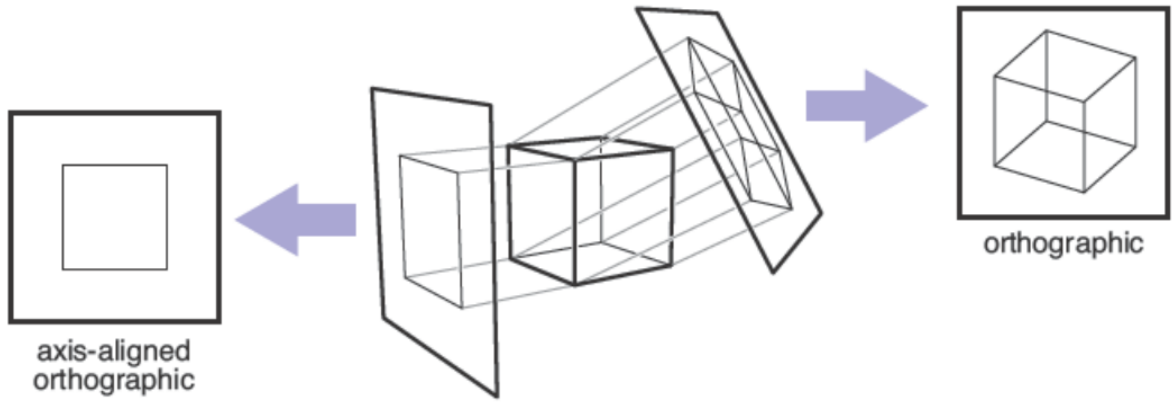


图 4.3 当投影线彼此平行，且都垂直于像平面时，生成的视图称为正交视图。图内 axis-aligned orthographic 表示“沿坐标轴对齐的正交视图”，orthographic 表示“正交视图”。

原书边注：有些书籍把“正交（orthographic）”一词专门用于投影方向平行于某条坐标轴的情况。

平行投影经常用于机械制图和建筑制图，因为它能让原本平行的直线在图像中继续保持平行，而且可以保留那些与像平面平行的平面物体的尺寸和形状。

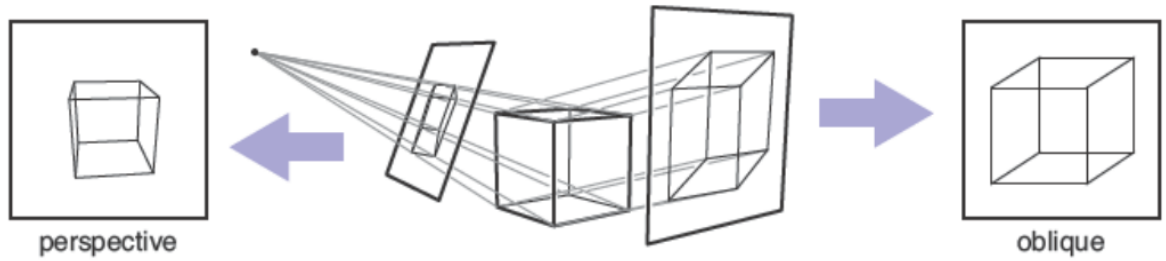


图 4.4 当平行投影的像平面与投影方向形成一定角度时，这种投影称为斜投影（右）。在透视投影中，所有投影线都经过同一个观察点，而不是彼此平行（左）。图中所示的透视视图并非斜视图，因为经过图像中心的一条投影线垂直于像平面。图内 perspective 为透视投影，oblique 为斜投影。

平行投影的优点，同时也是它的局限。在日常生活中（在照片中尤其明显），物体离我们越远，看起来就越小；因此，朝远方延伸的平行直线，在视觉上并不会显得平行。这是因为眼睛和相机并不是只接收来自某一个固定观察方向的光，而是接收所有经过某个特定观察点的光。

正如文艺复兴时期以来的艺术家们所认识到的那样，我们可以通过透视投影（perspective projection）得到看起来自然的视图：只需沿着经过同一点（即观察点）的直线进行投影，而不是沿着彼此平行的直线投影（图 4.4）。这样一来，距离观察点越远的物体，投影之后自然就越小。透视视图由观察点（而不是投影方向）和像平面共同决定。与平行投影类似，透视视图也分为斜视与非斜视两种；区分依据是图像中心处的投影方向。

你可能学过绘画中的三点透视（three-point perspective）规则。这是一套用于手工绘制透视图的方法（图 4.5）。关于透视，有一个令人惊讶的事实：只要遵循其背后那个简单的数学规则，透视绘图的全部规则就会自动得到满足。这个规则是：把物体沿着指向眼睛的直线进行投影，然后在这些投影线与眼睛前方的视平面相交的位置绘制物体。

第 4 章：光线追踪（4.3—4.3.1）

《Fundamentals of Computer Graphics》第 5 版 · 中文译本 · 第三十九批

翻译范围：原书第 82—85 页 · 4.3 导言与 4.3.1 · 保留图 4.5—4.9、全部公式、伪代码和原书边注

4.3 观察射线的计算（Computing Viewing Rays）

从上一节可以看出，生成射线所需的基本要素是观察点（viewpoint；对于平行视图，则是观察方向）和像平面（image plane）。确定相机几何关系的具体方法有很多种。本节介绍一种基于标准正交基（orthonormal basis）的方法，它能够支持常规视图以及倾斜的平行视图与正交视图。

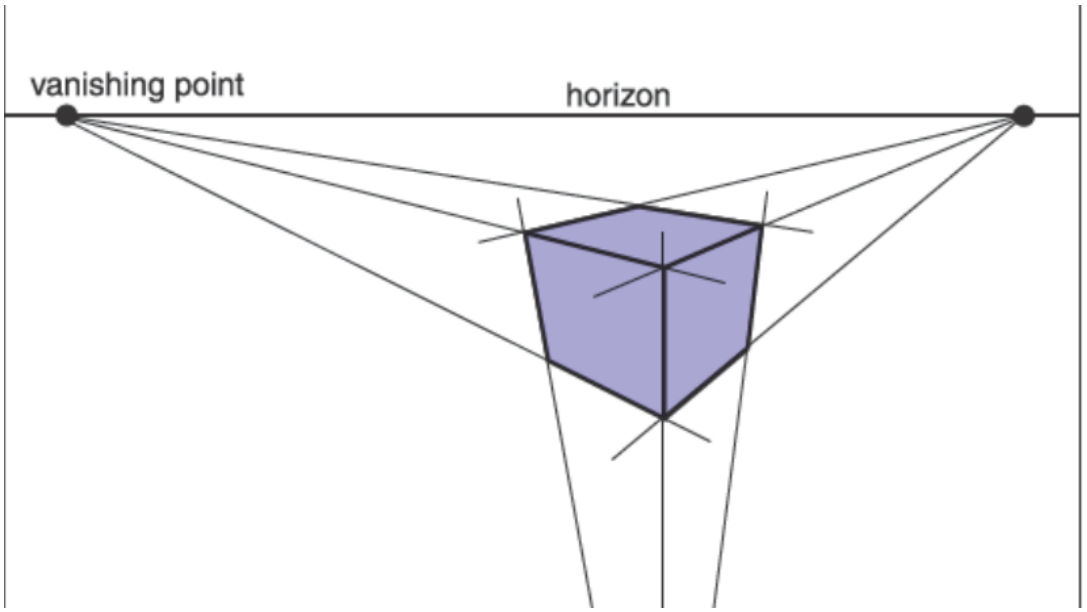


图 4.5 在三点透视中，画家会选取一些“消失点”，让平行线在这些点汇聚。彼此平行的水平线会在地平线上的某个点相交。每组平行线都有属于自己的消失点。只要依据正确的几何原理实现透视，这些规则便会自动得到遵循。图内 vanishing point 表示“消失点”，horizon 表示“地平线”。（原书将此图排在本节开头，用于说明上一节的三点透视。）

为了生成射线，我们首先需要用数学形式表示射线。射线实际上只包含一个起点和一个传播方向，因此用三维参数直线来表示非常合适。正如第 2.7.7 节所讨论的，从眼睛的位置 e 出发、经过像平面上的一点 s 的三维参数直线（图 4.6）可写为：

$$\mathbf{p}(t) = \mathbf{e} + t(\mathbf{s} - \mathbf{e})$$

这个式子应理解为：“从 e 出发，沿向量 s − e 前进相当于该向量长度的 t 倍的距离，便得到点 p。”因此，给定参数 t，就能确定相应的点 p。其中，e 是射线的起点，s − e 是射线的方向向量。

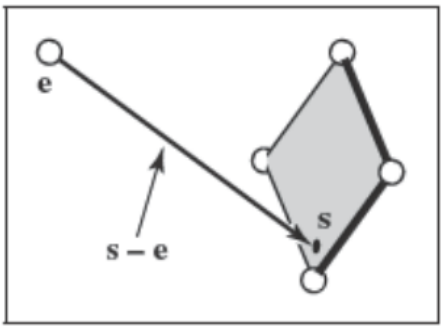


图 4.6 从眼睛出发，经过像平面上一点的射线。

注意，p(0) = e，p(1) = s。更一般地说，如果 0 < t⊠ < t⊠，那么 p(t⊠) 比 p(t⊠) 更接近眼睛。此外，如果 t < 0，那么 p(t) 就位于眼睛的“后方”。这些性质会在稍后寻找射线所击中的、位于眼睛前方的最近物体时发挥作用。

原书边注：请注意，这里重复使用了变量 t：一处用它表示射线参数；另一处用它表示图像上边界沿 v 方向的坐标。

在程序中，射线总是用某种结构体或对象表示，用来存储射线的位置和方向。例如，在面向对象的程序中，可以写成：

```
class Ray
 Vec3 o // 射线起点
 Vec3 d // 射线方向
 Vec3 evaluate(real t)
 return o + t * d
```

这里假设存在一个表示三维向量的 Vec3 类，并且它支持通常的向量算术运算。

为了计算观察射线，我们需要知道 e（它是已给定的）以及 s。求取 s 乍看有些困难，但只要选择合适的坐标系，这个问题实际上很直接。

我们介绍的全部射线生成方法，都从一个称为相机坐标架（camera frame）的标准正交坐标架出发（图 4.7）。这个坐标架由眼睛的位置 e（即观察点）和三个基向量 u、v、w 构成：从相机的视角看，u 指向右方，v 指向上方，w 指向后方，因此 {u, v, w} 形成一个右手坐标系。

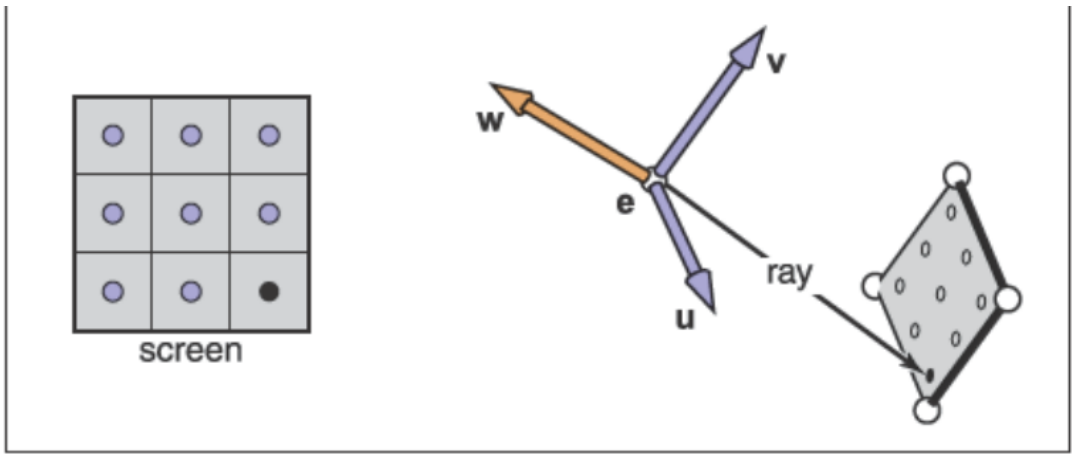


图 4.7 屏幕上的采样点被映射到三维成像窗口上排列方式相似的一组点。我们向每一个这样的点发射一条观察射线。图内 screen 为“屏幕”，ray 为“射线”。

构造相机坐标架最常见的方法，是使用观察点、观察方向（view direction）以及向上向量（up vector）。观察点直接作为 e，观察方向为 −w；向上向量则用于构造一个正交基，使 v 与 w 落在由观察方向和向上方向共同确定的平面中。具体过程可参见第 2.4.7 节“由两个向量构造标准正交基”（图 4.8）。

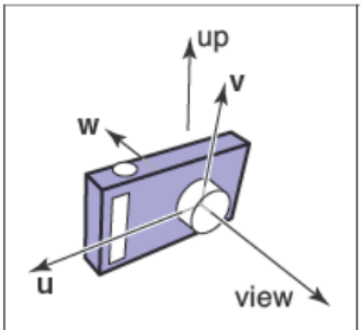


图 4.8 相机坐标架的向量，以及观察方向和向上方向。向量 w 与观察方向相反；v、w 和向上向量共面。图内 view direction 表示“观察方向”，up direction 表示“向上方向”。

原书边注：由于 v 与 w 必须互相垂直，向上向量和 v 通常并不完全相同。不过，把向上向量指定为场景中竖直朝上的方向，通常就能使相机保持人们所理解的“正立”姿态。

4.3.1 正交视图（Orthographic Views）

对于正交视图，所有射线的方向都是 −w。尽管平行视图本身并不存在一个真正的观察点，我们仍然可以使用相机坐标架的原点，来定义射线出发的平面。这样，也就可以区分哪些物体位于相机的后方。

观察射线应当从由点 e 与向量 u、v 确定的平面上出发。剩下需要确定的，只有图像在这个平面中的具体位置。我们使用四个数来表示图像的四条边界：l 和 r 分别是沿 u 方向从 e 测量的图像左边界和右边界坐标；b 和 t 分别是沿 v 方向从 e 测量的图像下边界和上边界坐标。通常满足 $l < 0 < r$，$b < 0 < t$（见图 4.9a）。

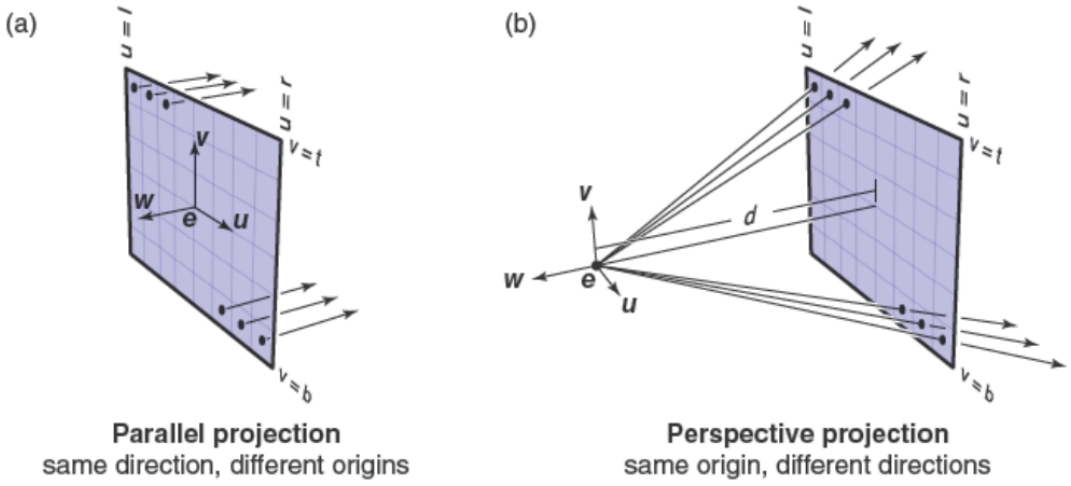


图 4.9 利用相机坐标架生成射线。(a) 在正交视图中，射线从像平面上各像素的位置出发，并且方向完全相同，均等于观察方向；(b) 在透视视图中，射线均从观察点出发，每条射线的方向由观察点 e 与像平面上对应像素的位置之间的连线确定。图内 parallel projection 表示“平行投影”，perspective projection 表示“透视投影”；前者“方向相同，起点不同”，后者“起点相同，方向不同”。

原书边注：直观上，正交观察射线似乎应当从无限远处发出；但如果这样做，例如，就不可能生成从房间内部观察物体的正交视图。

在第 3.2 节中，我们讨论过图像的像素坐标。为了把包含 $n_x \times n_y$ 个像素的图像放入一个大小为 (r − l) × (t − b) 的矩形中，像素的水平间距应为 $(r - l)/n_x$，垂直间距应为 $(t - b)/n_y$。与此同时，图像四周还应留出半个像素间距，以便使像素网格居中放置在图像矩形内。因此，光栅图像中位置为 (i, j) 的像素，对应以下坐标：

$$u = l + \frac{(r-l)(i+0.5)}{n_x}$$

$$v = b + \frac{(t-b)(j+0.5)}{n_y} \qquad (4.1)$$

这里，(u, v) 是像素在像平面中的坐标，以点 e 为原点、以基向量 {u, v} 为参照来测量。请注意，此处的普通字母 u、v 是两个标量坐标；作为基向量时，u、v 则表示三维向量，二者应根据上下文区分。

原书边注：许多系统假设 l = −r 且 b = −t，因此只需指定图像的宽度和高度。

在正交视图中，我们可以直接把像素在像平面上的位置作为射线起点；射线方向则已经确定，就是观察方向。因此，生成正交观察射线的过程是：

```
使用式 (4.1) 计算标量坐标 u、v
ray.o ← e + u * u + v * v
ray.d ← −w
```

原书边注：同时指定 l 和 r 会带来冗余：如果把观察点向右移动一点，并相应减小 l 和 r，最终视图不会改变（沿 v 轴也是一样）。

要得到倾斜的平行视图也很简单：只需允许把像平面的法线 w 与观察方向 d 分开指定。除此之外，生成过程完全相同，只要将 −w 替换成 d 即可。当然，构造基向量 u 和 v 时，仍然要使用 w。

第 4 章：光线追踪（4.3.2）

《Fundamentals of Computer Graphics》第 5 版 • 中文译本 • 第四十批

翻译范围：原书第 86 页 • 第 4.3.2 节全部正文与射线生成伪代码 • 另附上一批中的原书图 4.9，便于独立阅读

4.3.2 透视视图（Perspective Views）

对于透视视图（perspective view），所有射线都具有相同的起点，即观察点（viewpoint）；不同像素所对应的射线，区别在于它们各自的方向。

此时，像平面不再位于 e 处，而是位于 e 前方、与 e 相距 d 的位置。这个距离称为像平面距离（image plane distance），人们也常常不太严格地称它为“焦距”（focal length），因为选择 d 的作用与在真实相机中选择焦距的作用相同。

每条射线的方向，都由观察点以及对应像素在像平面上的位置共同确定。图 4.9 展示了这种情形。生成透视观察射线的步骤与正交视图类似：

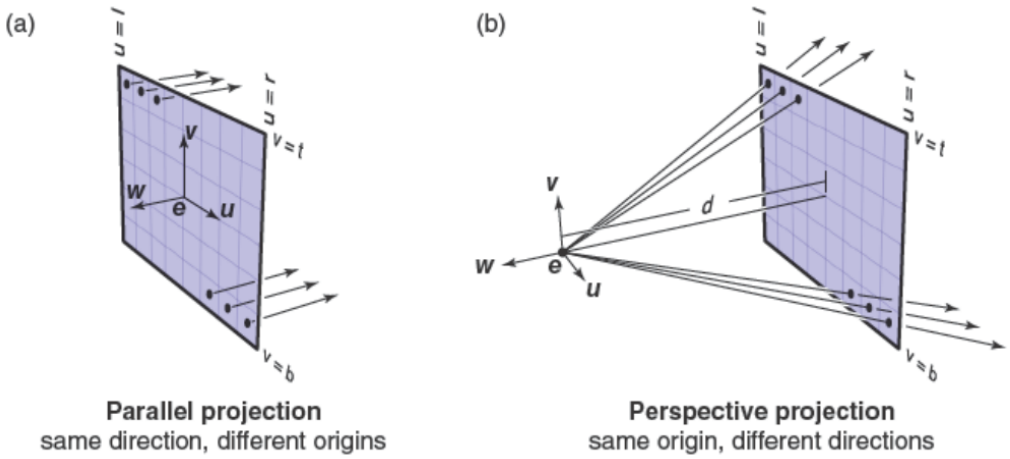


图 4.9 使用相机坐标架生成射线。（a）正交视图：射线的方向相同，起点不同。（b）透视视图：射线都从观察点 e 出发，各自经过像平面上对应的像素位置，因此起点相同，方向不同。此图沿用上一批，供本节对照阅读。

```
使用式 (4.1) 计算标量坐标 u 和 v
ray.o ← e
ray.d ← −d * w + u * u + v * v
```

其中，d 是观察点到像平面的距离；u 和 v 是式（4.1）算出的像素二维坐标；右侧加粗的 u、v、w 则表示相机坐标架的三个基向量。与正交视图不同，透视视图的射线起点始终为 e，而方向会随着像素位置改变。

$$\text{ray.d} = -d\mathbf{w} + u\mathbf{u} + v\mathbf{v}$$

与平行投影类似，只要把像平面的法线方向与投影方向分开指定，也可以得到倾斜的透视视图（oblique perspective view）。

第 4 章：光线追踪（4.4—4.4.1）

《Fundamentals of Computer Graphics》第 5 版 · 中文译本 · 第四十一批

翻译范围：原书第 86—87 页 · 4.4 导言与 4.4.1《射线与球体求交》全部正文和数学公式

4.4 光线与物体求交（Ray-Object Intersection）

生成射线 e + td 之后，接下来需要找到射线与任意物体的第一个交点，并要求 $t > 0$。在实际应用中，解决一个稍微更一般化的问题会更有用：寻找射线与某个表面之间、参数 t 位于区间 $[t_0, t_1]$ 内的第一个交点。

因此，最基本的射线求交问题，就是 $t_0 = 0$、$t_1 = +\infty$ 时的特殊情况。我们将分别针对球体和三角形求解这一问题。下一节还会讨论多个物体的情况。

4.4.1 射线与球体求交（Ray-Sphere Intersection）

已知一条射线 p(t) = e + td，以及由隐式方程 f(p) = 0 定义的曲面（参见第 2.7.3 节），我们希望确定它们在哪里相交。当射线上的某个点满足曲面的隐式方程时，就出现了交点。因此，我们要寻找的参数 t，就是满足下式的解：

$$f(\mathbf{p}(t)) = 0 \quad \text{or} \quad f(\mathbf{e} + t\mathbf{d}) = 0$$

对于一个以 $c = (x_c, y_c, z_c)$ 为球心、半径为 R 的球体，可以用下面的隐式方程表示：

$$(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - R^2 = 0$$

我们也可以用向量形式写出同一个方程：

$$(\mathbf{p} - \mathbf{c}) \cdot (\mathbf{p} - \mathbf{c}) - R^2 = 0$$

凡是满足这个方程的点 p，都位于球面上。如果把射线上的点 p(t) = e + td 代入上述球面方程，就会得到一个以 t 为未知数的方程。满足此方程的 t 值，正是那些会使射线上的点落在球面上的参数值：

$$(\mathbf{e} + t\mathbf{d} - \mathbf{c}) \cdot (\mathbf{e} + t\mathbf{d} - \mathbf{c}) - R^2 = 0$$

整理各项，可以得到：

$$(\mathbf{d} \cdot \mathbf{d})t^2 + 2\mathbf{d} \cdot (\mathbf{e} - \mathbf{c})t + (\mathbf{e} - \mathbf{c}) \cdot (\mathbf{e} - \mathbf{c}) - R^2 = 0$$

此时，除了参数 t 以外，其他所有量都是已知的。因此，这是一个关于 t 的典型一元二次方程，具有以下形式：

$$At^2 + Bt + C = 0$$

第 2.2 节讨论了这个方程的解法。在一元二次方程的求根公式中，根号内的 $B^2 - 4AC$ 称为判别式（discriminant），它能够告诉我们有多少个实数解。

- 如果判别式小于零，开平方后得到虚数，因此直线与球体没有交点。
- 如果判别式大于零，就有两个解：一个对应射线进入球体的位置，另一个对应射线离开球体的位置。
- 如果判别式等于零，射线与球面相切，恰好只接触一个点。

把球体求交方程中的实际各项代入求根公式，并约去公因子 2，就得到：

$$t = \frac{-\mathbf{d}\cdot(\mathbf{e}-\mathbf{c}) \pm \sqrt{(\mathbf{d}\cdot(\mathbf{e}-\mathbf{c}))^2 - (\mathbf{d}\cdot\mathbf{d})((\mathbf{e}-\mathbf{c})\cdot(\mathbf{e}-\mathbf{c}) - R^2)}}{\mathbf{d}\cdot\mathbf{d}}$$

在实际实现时，应当先检查判别式的值，再计算其他各项。为了正确找出位于区间 $[t_0, t_1]$ 内的最近交点，需要考虑三种情况：如果两个解中较小的那个落在该区间内，那么它就是第一个交点；否则，如果较大的那个解位于区间内，那么它就是第一个交点；如果两者都不在区间内，就说明该区间内没有交点。

正如第 2.7.4 节所讨论的，在球面上的点 p 处，其法向量可由梯度给出：

$$\mathbf{n} = 2(\mathbf{p}-\mathbf{c})$$

对应的单位法向量为：

$$\hat{\mathbf{n}} = \frac{\mathbf{p}-\mathbf{c}}{R}$$

第 4 章：光线追踪（4.4.2）

《Fundamentals of Computer Graphics》第 5 版 • 中文译本 • 第四十二批

翻译范围：原书第 88—90 页 • 4.4.2 全部正文、图 4.10、重心坐标、克拉默法则公式及伪代码

4.4.2 射线与三角形求交（Ray-Triangle Intersection）

计算射线与三角形交点的方法有很多种。这里介绍一种利用包含该三角形的参数平面上的重心坐标（barycentric coordinates）进行求交的方法。之所以选择它，是因为除了三角形的顶点之外，该方法不需要长期存储其他数据（Snyder & Barr，1987）。

为了求出射线与参数曲面的交点，我们建立一个方程组，要求两者在笛卡儿坐标系下的各个坐标分量都相等：

$$\left.\begin{aligned} x_e + t x_d &= f(u,v) \\ y_e + t y_d &= g(u,v) \\ z_e + t z_d &= h(u,v) \end{aligned}\right\} \quad \text{or,} \quad \mathbf{e} + t\mathbf{d} = \mathbf{f}(u,v).$$

这里有三个方程、三个未知数（t、u 和 v）。如果曲面是一个参数平面，那么其参数方程是线性的，可以如第 2.9.2 节所述写成向量形式。设三角形的顶点分别为 a、b 和 c，则交点必须满足：

$$\mathbf{e} + t\mathbf{d} = \mathbf{a} + \beta(\mathbf{b} - \mathbf{a}) + \gamma(\mathbf{c} - \mathbf{a}) \qquad (4.2)$$

其中，t、β 和 γ 是待求的数。解这个方程不仅能求得 t，从而确定交点沿射线的位置，还能求得 (β, γ)，从而确定交点相对于三角形的位置。图 4.10 所示的交点 p，就是 e + td。

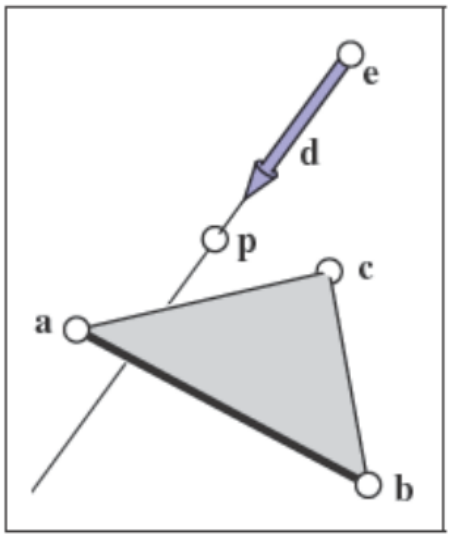


图 4.10 射线在点 p 处击中包含三角形的平面。图中 e 是射线起点，d 是射线方向，a、b、c 是三角形的三个顶点。

由第 2.9.2 节可知，交点位于三角形内部，当且仅当 β > 0、γ > 0，并且 β + γ < 1。否则，射线与三角形所在平面的交点会落在三角形外部，因此射线并未击中三角形。如果方程组无解，则要么三角形退化，要么射线与三角形所在的平面平行。

为了求解式（4.2）中的 t、β 和 γ，我们把向量形式的方程展开为三个坐标分量的方程：

$$
\begin{aligned}
x_e + t x_d &= x_a + \beta(x_b - x_a) + \gamma(x_c - x_a) \\
y_e + t y_d &= y_a + \beta(y_b - y_a) + \gamma(y_c - y_a) \\
z_e + t z_d &= z_a + \beta(z_b - z_a) + \gamma(z_c - z_a)
\end{aligned}
$$

这可以改写为一个标准的线性方程组：

$$
\begin{bmatrix}
x_a - x_b & x_a - x_c & x_d \\
y_a - y_b & y_a - y_c & y_d \\
z_a - z_b & z_a - z_c & z_d
\end{bmatrix}
\begin{bmatrix}
\beta \\ \gamma \\ t
\end{bmatrix}
=
\begin{bmatrix}
x_a - x_e \\ y_a - y_e \\ z_a - z_e
\end{bmatrix}.
$$

求解这个 3 × 3 线性方程组，经典方法中最快的一种是克拉默法则（Cramer’s rule）。它给出以下解：

$$
\beta = \frac{\begin{vmatrix} x_a - x_e & x_a - x_c & x_d \\ y_a - y_e & y_a - y_c & y_d \\ z_a - z_e & z_a - z_c & z_d \end{vmatrix}}{|\mathbf{A}|},
$$

$$
\gamma = \frac{\begin{vmatrix} x_a - x_b & x_a - x_e & x_d \\ y_a - y_b & y_a - y_e & y_d \\ z_a - z_b & z_a - z_e & z_d \end{vmatrix}}{|\mathbf{A}|},
$$

$$
t = \frac{\begin{vmatrix} x_a - x_b & x_a - x_c & x_a - x_e \\ y_a - y_b & y_a - y_c & y_a - y_e \\ z_a - z_b & z_a - z_c & z_a - z_e \end{vmatrix}}{|\mathbf{A}|},
$$

其中，矩阵 A 为：

$$
\mathbf{A} = \begin{bmatrix} x_a - x_b & x_a - x_c & x_d \\ y_a - y_b & y_a - y_c & y_d \\ z_a - z_b & z_a - z_c & z_d \end{bmatrix},
$$

而 |A| 表示矩阵 A 的行列式（determinant）。这些 3 × 3 行列式具有共同的子表达式，在实际编程时可以利用它们提高计算效率。考虑使用占位变量表示的下列线性方程组：

$$\begin{bmatrix} a & d & g \\ b & e & h \\ c & f & i \end{bmatrix} \begin{bmatrix} \beta \\ \gamma \\ t \end{bmatrix} = \begin{bmatrix} j \\ k \\ l \end{bmatrix},$$

> 译者提示：这里矩阵中的 a、b、c 等小写字母只是表示矩阵元素的占位变量，不再专指前面三角形的顶点；它们与未知数 β、γ、t 的含义也不相同。

根据克拉默法则，我们有：

$$\beta = \frac{j(ei - hf) + k(gf - di) + l(dh - eg)}{M}$$
$$\gamma = \frac{i(ak - jb) + h(jc - al) + g(bl - kc)}{M}$$
$$t = -\frac{f(ak - jb) + e(jc - al) + d(bl - kc)}{M}$$

其中：

$$M = a(ei - hf) + b(gf - di) + c(dh - eg)$$

通过复用诸如“ei − hf”这样的中间计算结果，我们可以减少运算次数。

在射线与三角形求交算法中，求解线性方程组的过程还可以加入一些提前终止的条件。因此，函数可以写成如下形式：

```
boolean raytri(Ray r, vector3 a, vector3 b, vector3 c,
 interval [t0, t1])
 compute t
 if (t < t0) or (t > t1) then
 return false
 compute γ
 if (γ < 0) or (γ > 1) then
 return false
 compute β
 if (β < 0) or (β > 1 − γ) then
 return false
 return true
```

第 4 章：光线追踪（4.4.3）

《Fundamentals of Computer Graphics》第 5 版 • 中文译本 • 第四十三批

翻译范围：原书第 90—91 页 • 4.4.3 全部正文、Surface 与 HitRecord 接口和伪代码

4.4.3 软件中的射线求交（Ray Intersection in Software）

编写光线追踪程序时，采用面向对象的设计通常是个不错的选择。例如，可以定义一个名为 Surface 的类，再让 Triangle、Sphere 等类继承它。任何能够与射线相交的对象，都应当是 Surface 的子类；这不仅包括单个几何表面，也包括一组表面，以及用于提高效率的数据结构（见第 12.3 节）。

这样，光线追踪程序只需要持有一个指向 Surface 的引用，就能够表示整个模型；而且，后续可以透明地添加新的物体类型和加速结构，不必改变使用这个接口的代码。

Surface 类最关键的接口，是用于计算射线与表面交点的方法（Kirk & Arvo，1988）：

```
class Surface
 HitRecord hit(Ray r, real t0, real t1)
```

这里，(t_0, t_1) 是射线上允许返回交点的参数区间；HitRecord 则是一个类，负责保存后续处理表面交点时所需要的全部数据：

```
class HitRecord
 Surface s | 被射线击中的表面
 real t | 交点沿射线的参数值
 Vec3 n | 交点处的表面法向量
 …
```

至少需要记录被击中的表面、t 的值以及表面法向量。不过，也可以存储其他数据，例如纹理坐标或切向量。

具体采用哪一种语言，会影响交点记录的返回方式：HitRecord 不一定需要作为函数返回值直接返回；也可以通过引用把记录传入函数，再由函数填写记录中的各个字段。

如果一次射线求交未击中任何物体，可以通过一个满足 $t = \infty$ 的交点记录来表示“未命中”。

第 4 章：光线追踪（4.4.4）

《Fundamentals of Computer Graphics》第 5 版 • 中文译本 • 第四十四批

翻译范围：原书第 91 页 • 4.4.4 全部正文、Group 求交伪代码及图 4.11

4.4.4 与一组物体求交（Intersecting a Group of Objects）

显然，大多数有趣的场景都不止包含一个物体。当我们计算射线与整个场景的交点时，必须只找出沿射线方向最接近相机的那个交点。

一种简单的实现思路，是把一组物体本身也视为另一种物体。为了计算射线与这一组物体的交点，只需要分别求出射线与组内各个物体的交点，然后返回参数 t 最小的那个交点。下面的代码会检测参数 t 位于区间 $[t_0, t_1]$ 内的交点：

```
class Group, subclass of Surface
 list-of-Surface surfaces | 组内全部表面的列表
 HitRecord hit(Ray ray, real t0, real t1)
 HitRecord closest-hit(∞) | 初始化为“未命中”
 for surf in surfaces do
 rec = surf.hit(ray, t0, t1)
 if rec.t < ∞ then
 closest-hit = rec
 t1 = t
 return closest-hit
```

请注意，这段代码会逐步缩小求交区间 $[t_0, t_1]$。这样，后续对 surf.hit 的调用，只会命中那些比目前已知的最近交点还要靠近相机的表面。

> 译者提示（非原书正文）：原书伪代码在更新最近交点后写的是 $t_1 = t$，但此前没有在该代码段中单独赋值变量 t。结合上下文，此处的意图应是用当前命中的 rec.t 更新区间上界；上方仍保留原书的写法。

一旦射线与场景求交的功能正确工作，我们就能渲染出类似图 4.11 的图像。不过，要得到更好看的结果，还需要加入更多视觉线索，接下来我们就会讨论这些内容。



图 4.11 一个仅使用射线生成和表面求交、但尚未进行着色的简单场景。每个像素只根据射线击中的物体，设置为一种固定颜色。

第 4 章：光线追踪（4.5—4.5.1）

《Fundamentals of Computer Graphics》第 5 版 • 中文译本 • 第四十五批

翻译范围：原书第 91—92 页 • 4.5 导言与 4.5.1 全部正文、光源分类、四项几何量与公式

4.5 着色（Shading）

当我们已经知道某个像素所对应的可见表面时，就可以通过计算着色模型（shading model），得到这个像素的数值。具体如何计算，完全取决于应用需求：可用的方法既包括简单的经验性规则，也包括精细的、基于物理的模型。完全相同的着色模型既可以用于光线追踪，也可以用于按照物体顺序进行渲染的方法（object-order rendering）。

第 5 章会介绍一种适合基础光线追踪器的简单着色模型；本章中的渲染图像采用的就是这一模型。如果想获得更高的真实感，可以进一步使用第 14 章讨论的模型，它们更忠实于真实物体表面的物理性质。本节将讨论光线追踪器如何计算着色所需的输入。

4.5.1 光源（Light Sources）

为了实现着色，光线追踪程序总是需要维护一个光源列表。对于第 5 章介绍的着色模型，我们需要三种光源：

- 点光源（point light）：从空间中的某一个点发出光。
- 方向光（directional light）：从同一个方向照亮整个场景。
- 环境光（ambient light）：提供恒定的照明，用来填补阴影区域。

功能更丰富的系统还会支持其他光源类型，例如面积光源（area light，基本上就是能够发光的场景几何体），或者环境照明光源（environment light，它通过一张图像来表示来自天空等遥远光源的光）。

计算点光源或方向光所产生的着色，需要某些几何信息。在光线追踪器中，一旦确定观察射线击中了一个表面，我们就已经具备了求出以下四个量所需的全部信息：

- 着色点 x（shading point）：把交点的参数 t 代入观察射线方程，即可计算出该点的位置。
- 表面法向量 n（surface normal）：它取决于表面的类型（球体、三角形等）。每一种表面都必须能够计算出射线交点处的法向量。
- 光照方向 l（light direction）：在执行着色计算时，根据光源的位置或方向计算出来。
- 观察方向 v（viewing direction）：它与观察射线的方向正好相反；如果射线方向为 d，那么：

$$\mathbf{v} = -\frac{\mathbf{d}}{\|\mathbf{d}\|}$$

来自环境光源的着色计算要简单得多：因为光可以视为来自四面八方，所以不需要光照方向 l；着色结果也不依赖观察方向 v。对于第 5 章中的简单模型，它甚至不依赖着色点 x 或表面法向量 n。

当一个场景包含多个光源时，计算着色只需要把各个光源的贡献相加。在基础光线追踪器中，可以直接遍历光源列表，分别计算每个光源产生的着色，再将结果累加到像素颜色中。

第 4 章：光线追踪（4.5.2）

《Fundamentals of Computer Graphics》第 5 版 · 中文译本 · 第四十六批

翻译范围：原书第 92—94 页 · 4.5.2 全文、四组伪代码及图 4.12

4.5.2 软件中的着色（Shading in Software）

光线追踪程序通常包含用于表示光源和材质的对象。光源可以是 Light 类的各个子类的实例，而且必须包含足以完整描述相应光源的信息。由于着色还需要描述表面材质的参数，因此另一个有用的类是 Material：它封装了计算着色模型所需的一切信息。

不同的系统会以不同方式在光源和材质之间划分着色计算的职责。一种与本章讲述方式相符的做法，是让光源负责总体光照计算，而让材质负责计算 BRDF（双向反射分布函数）的值。按照这种设计，这些类的接口可以写成：

```
class Light
  Color illuminate(Ray ray, HitRecord hrec)


class Material
  Color evaluate(Vec3 l, Vec3 v, Vec3 n)
```

这样，每个表面都要保存一个指向其材质的引用。采用这种方式，点光源的照明可以实现为：

```
class PointLight，Light 的子类
  Color I
  Vec3 p
  Color illuminate(Ray ray, HitRecord hrec)
    Vec3 x = ray.evaluate(hrec.t)
    real r = ||p − x||
    Vec3 l = (p − x) / r
    Vec3 n = hrec.normal
    Color E = max(0, n · l) I / r^2
    Color k = hrec.surface.material.evaluate(l, v, n)
    return kE
```

这些计算假定 Color 类保存颜色的 RGB 分量，并支持逐分量乘法。这样的组织方式也便于把环境光照当作一种光源来处理：只需让环境光系数成为材质的一个属性即可。

```
class AmbientLight，Light 的子类
  Color I_a
  Color illuminate(Ray ray, HitRecord hrec)
    Color k_a = hrec.surface.material.k_a
    return k_a I_a
```

为一条射线执行完整的着色计算，包括求交和处理多个光源，可以写成下面这样：

```
function shade-ray(Ray ray, real t_0, real t_1)
  HitRecord rec = scene.hit(ray, t_0, t_1)
  if rec.t < ∞ then
    Color c = 0
    for light in scene.lights do
      c = c + light.illuminate(ray, rec)
    return c
  else
    return background-color
```

这种组织方式使材质与光源保持了相当程度的分离，也使你日后能够在不影响现有用法的情况下，方便地添加新的材质类型和光源类型。纹理会给光线追踪器的架构增加一些复杂性；参见第 11.2.5 节。

单靠着色，就能使三维物体的图像更加逼真，也更容易理解；但着色本身并不能展现物体与其他物体之间的相互作用。例如，图 4.12 中的球体虽然放在地板上，却显得像是悬浮在地板上方。

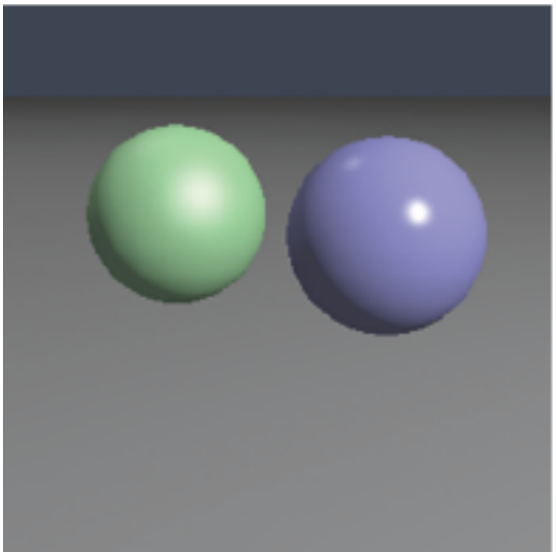


图 4.12 使用第 5 章介绍的着色模型，并由两个点光源照明而渲染出的简单场景。

译注（非原书正文）：上面的 PointLight 伪代码使用了观察方向 v，但没有在函数内部显式给出它的定义。根据前一小节，若观察射线的方向为 d，则 v = −d / ||d||。

第 4 章：光线追踪（4.5.3）

《Fundamentals of Computer Graphics》第 5 版 • 中文译本 • 第四十七批

翻译范围：原书第 94—95 页 • 4.5.3 全部正文、阴影射线伪代码与图 4.13—4.15

4.5.3 阴影（Shadows）

一旦在光线追踪器中实现了基本着色，就可以非常容易地为点光源和方向光添加阴影。设想我们处在某个正在着色的表面点 x 上：如果朝着光源的方向“看去”，发现自己与光源之间有其他物体，那么这个点就处在阴影中。如果两者之间没有其他物体，光就没有被遮挡。

如图 4.13 所示，射线 $x + t\,l$ 没有与任何物体相交，因此点 x 不在阴影中。相反，点 x' 处于阴影中，因为射线 $x' + t\,l'$ 确实击中了一个物体。用于判断某个点是否处于阴影中的射线，称为阴影射线（shadow rays），以便与观察射线（viewing rays）相区分。

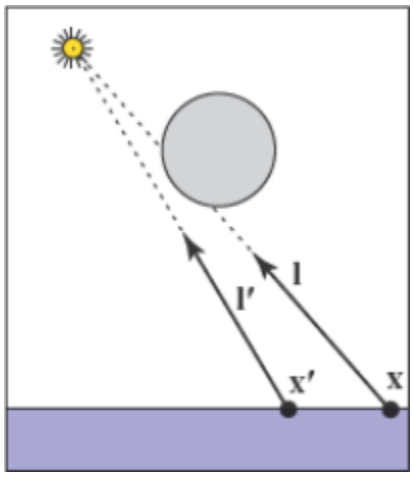


图 4.13 点 p 不处于阴影中，而点 q 处于阴影中。

要得到带阴影的着色算法，我们可以在计算某个光源所产生的着色贡献的代码中增加一条 if 判断，先确定光源是否被遮挡。在简单实现中，阴影射线会检查参数 t 位于 [0, r] 的区间；但由于数值计算不够精确，这可能使射线与点 p 所在的表面发生相交。为避免这种问题，通常把检测区间调整为 [ε, r]，其中 ε 是一个很小的正常数（图 4.14）。

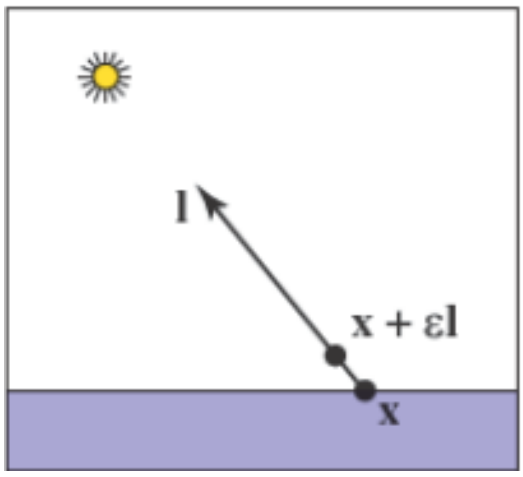


图 4.14 从 ε 开始测试射线的参数区间，可避免数值误差导致射线再次击中点 p 所在的表面。

在前面给出的 PointLight.illuminate 方法中，可以通过追踪一条阴影射线并增加条件判断来实现阴影测试：

```
HitRecord srec = scene.hit(Ray(x, l), ε, r)
if srec.t < ∞ then
  继续进行正常的光照计算
else
  return 0 | 着色点处于阴影中
```

方向光的阴影测试与此类似，但使用 $t_1 = \infty$，而不是 r。请注意：对每个光源进行照明计算，都需要一条独立的阴影射线；而计算环境光着色时，不需要进行阴影测试。

阴影在表现彼此靠近的物体之间的关系时具有重要的视觉作用，如图 4.15 所示。

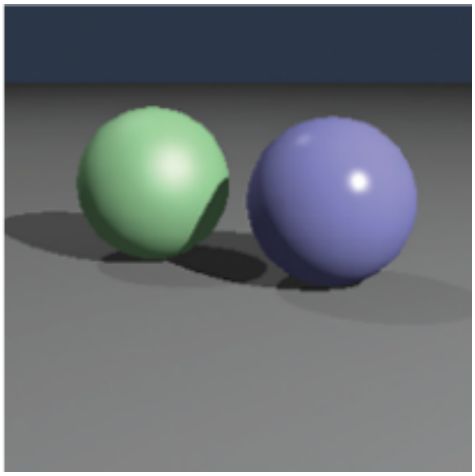


图 4.15 同一个场景，在两个点光源的照明下，使用着色和阴影渲染得到的效果。

译注（非原书正文）：原书图 4.13 的图内标记为 x 和 x′，但图注使用 p 和 q；两者指向的是同一对“未遮挡／被遮挡”的点。此外，原书所列伪代码的 if/else 分支与阴影判断规则相反：实际实现时，若 srec.t < ∞，说明阴影射线击中了遮挡物，应返回 0；没有命中时才应继续计算该光源的照明。这里保留了原书代码的写法，供对照。

第 4 章：光线追踪（4.5.4）

《Fundamentals of Computer Graphics》第 5 版 · 中文译本 · 第四十八批

翻译范围：原书第 95 页 · 4.5.4 全部正文、公式（4.3）、函数自身调用与图 4.16—4.17

4.5.4 镜面反射（Mirror Reflection）

在光线追踪程序中加入理想镜面反射（ideal specular reflection），或称镜面反射（mirror reflection），并不困难。关键的观察如图 4.16 所示：沿方向 e 观察的观看者，会看到从表面沿方向 r 望去的景象。向量 r 是向量 −d 关于表面法向量 n 的镜像，可以通过把 d 投影到表面法向量的方向上来计算：

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\mathbf{n} \tag{4.3}$$

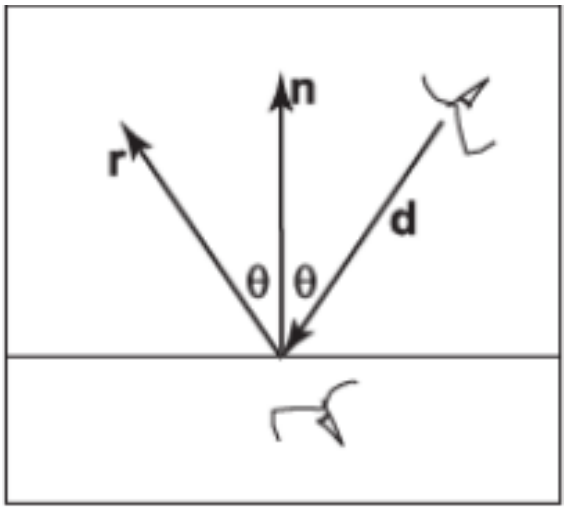


图 4.16 当观察者望向一面理想镜子时，沿 d 方向观看的人，会看到一个好像处于表面“下方”的观察者沿 r 方向所能看到的景象。

在现实世界中，光从表面反射时会损失一部分能量，而且不同颜色对应的能量损失可能并不相同。例如，黄金反射黄色光的效率高于反射蓝色光的效率，因此它会使被反射物体的颜色发生偏移。我们可以在 shade-ray 函数中增加一次对自身的调用（recursive call）来实现这种效果：在计算完所有光源的贡献之后，再加上一项：

$$\text{color } c = c + k_m \text{ shade-ray}(\text{Ray}(p, r), \varepsilon, \infty)$$

其中，k_m（m 表示“mirror reflection”，即镜面反射）是用于镜面反射的 RGB 颜色。我们还必须确保传入的 $t_0 = \varepsilon$；原因与前面使用阴影射线时相同：不能让反射射线击中产生它的那个物体。

上面这种自身调用存在一个问题：它可能永远不会终止。例如，如果一条射线从房间内部出发，就可能无限次地反射。通过规定最大调用层数（maximum recursion depth），可以解决这个问题。此外，只有当 k_m 不为零时才生成反射射线，也能提高代码的执行效率。

使用恒定的镜面反射系数 k_m，会使简单光线追踪器渲染出一种特有的外观（图 4.17）。但在现实世界中，这个系数会随着入射角而显著变化。关于更好的模型，请参见第 14 章。

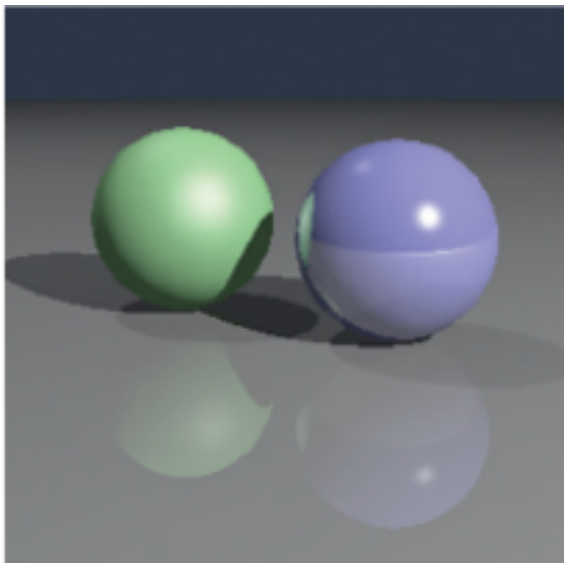


图 4.17 使用着色、阴影和镜面反射渲染的简单场景。地板和蓝色球体的镜面反射系数都不为零。

第 4 章：光线追踪（4.6）

《Fundamentals of Computer Graphics》第 5 版 · 中文译本 · 第四十九批

翻译范围：原书第 95—96 页 · 4.6 全部正文及文献引用

4.6 历史注释（Historical Notes）

光线追踪（ray tracing）在计算机图形学发展的早期就已经出现（Appel，1968），但直到计算能力足够强大时才得到较为广泛的应用（Kay & Greenberg，1979；Whitted，1980）。

与基本的按物体顺序渲染（object-order rendering）相比，光线追踪具有更低的渐近时间复杂度（asymptotic time complexity）（Snyder & Barr，1987；Muuss，1995；Parker et al.，1999；Wald，Slusallek，Benthin，& Wagner，2001）。尽管传统上光线追踪被视为一种离线方法，但实时光线追踪的实现正变得越来越普遍。

第 4 章：光线追踪（常见问题与习题）

《Fundamentals of Computer Graphics》第 5 版 • 中文译本 • 第五十批

翻译范围：原书第 96 页 • 常见问题 3 则、习题 3 道

常见问题（Frequently Asked Questions）

问题 1：为什么光线追踪中没有透视矩阵?

在使用 z 缓冲（z-buffer）的渲染方法中，透视矩阵的作用是让我们能够把透视投影转变为平行投影。光线追踪不需要这样做，因为只需让射线从观察点向外发散，就很容易隐式地实现透视投影。

问题 2：光线追踪能够实现交互式运行吗?

对于尺寸足够小的模型和图像，任何现代个人计算机都具有足够的计算能力，可以交互式地执行光线追踪。实际上，若要实现全屏幕的光线追踪，仍需要使用共用帧缓冲区的多个 CPU。计算机计算能力的增长速度远快于屏幕分辨率的增长速度，因此，普通个人计算机能够以屏幕分辨率对复杂场景进行光线追踪，只是时间早晚的问题。

问题 3：光线追踪在使用图形硬件的程序中有用吗?

光线追踪经常用于选取（picking）。当用户在三维图形程序中用指针单击某个像素时，程序需要确定哪个物体在该像素位置可见。光线追踪是完成这一判定的理想方法。

习题（Exercises）

1. 射线 (1, 1, 1) + t(−1, −1, −1) 与一个以原点为球心、半径为 1 的球体相交。求各交点对应的射线参数 t。注意：这是一个很好的程序调试测试用例。

2. 射线 (1, 1, 1) + t(−1, −1, −1) 与一个三角形相交，该三角形的三个顶点分别为 (1, 0, 0)、(0, 1, 0) 和 (0, 0, 1)。求交点的重心坐标（barycentric coordinates）以及对应的射线参数 t。注意：这也是一个很好的程序调试测试用例。

3. 对于“表现良好”的模型（即并非刻意构造为最坏情况的模型），通过粗略估算来求出光线追踪的大致时间复杂度。请分别分析预处理（preprocessing）阶段和图像计算（computing the image）阶段，以便预测对一个静态模型连续进行多帧光线追踪时的运行表现。